

COS 433: Cryptography

Instructor: Alex Lombardi, Office Hours Monday 4:15pm

TAs: Lacey Delucia, Fangqi Dong, Frederick Qiu (OH TBD)

Email (use this to reach us): cos433staff@princeton.edu

Course Website:



princeton-cos-433.github.io

What is cryptography?

What is cryptography?

Pre-1950: the art of secret writing

“Yhql ylgf ylf”

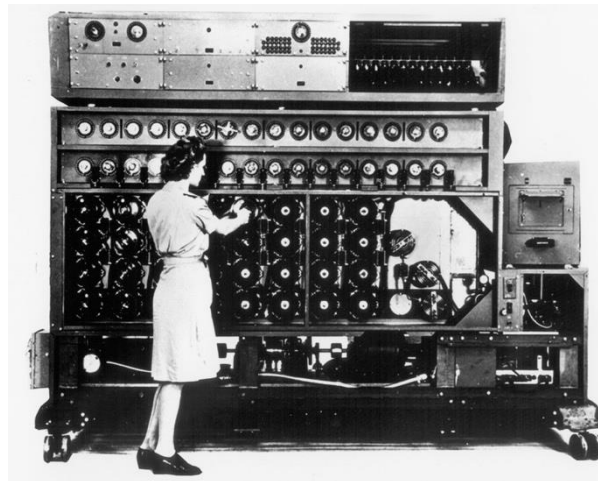
“Veni vidi vici”



"If he had anything confidential to say, he wrote it in cipher, that is, by so changing the order of the letters of the alphabet, that not a word could be made out. If anyone wishes to decipher these, and get at their meaning, he must substitute the fourth letter of the alphabet, namely D, for A, and so with the others."

—[Suetonius](#), [Life of Julius Caesar](#) 56

1900-1950: one of the first applications of (primitive) computers



What is cryptography?

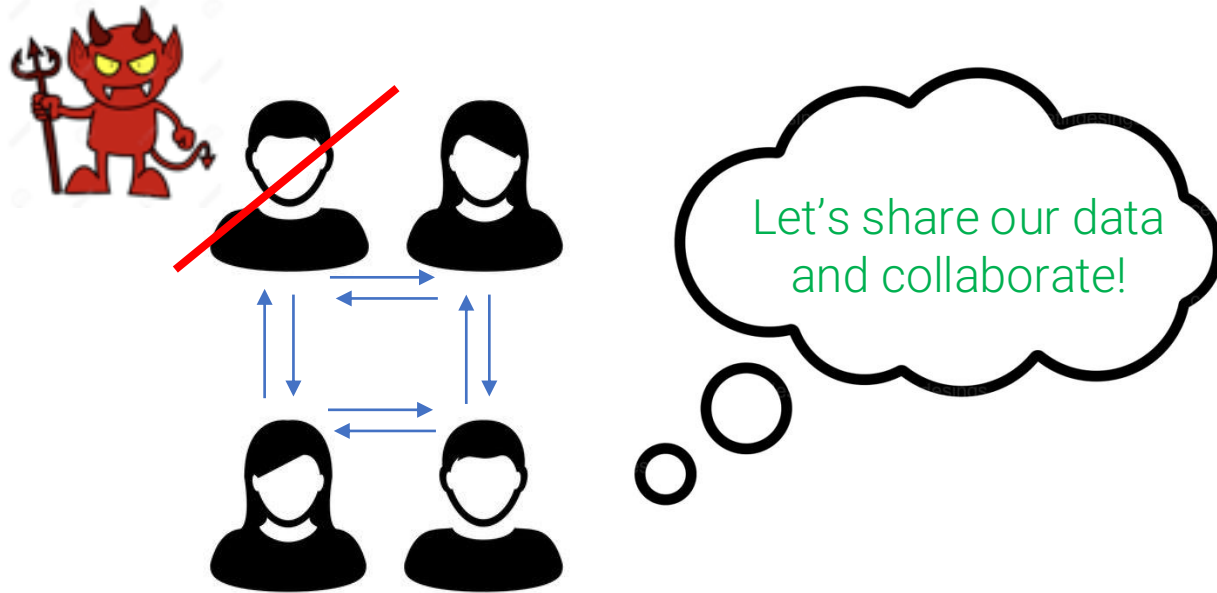
Modern definition:

The design and analysis of protocols that resist adversarial behavior.

What is cryptography?

Modern definition:

The design and **analysis** of **protocols** that **resist adversarial behavior**.



Theorem:  are
protected from 

What is cryptography?

Modern definition:

The design and **analysis** of **protocols** that **resist adversarial behavior**.

Another definition:

The **algorithmic** and **mathematical foundations** of **secure communication and computation**.

What is cryptography any good for?



Authorize a credit
card transaction...

without identity theft



Perform a massive,
private computation...

without running it locally

Course Information

Course Information

What you need in order to succeed:

- Comfort writing and understanding mathematical proofs
- Basic probability (expected value, variance)
- Asymptotic running time of an algorithm (like $O(n^2)$ vs. $O(2^n)$)

Helpful background: number theory (modular arithmetic), algorithms, NP hardness.

This course may be challenging! Come to office hours, ask questions (about the problem sets, lectures, etc.) on Ed, collaborate with your classmates.

Assignments

This course will have the following graded assignments:

- **Problem Set 0:** A short, one-week problem set at the beginning of the semester to help you calibrate your background.
- **Problem Sets 1–5:** Regular-length, biweekly problem sets.
- **Problem Set 6:** A cumulative problem set that will serve as your final.

Grading: Your assignments will be graded in two equally weighted parts:

1. Your written work.
2. Check-ins with the course staff. After Problem Sets 0, 3, and 6, you'll have a short individual meeting about your recent solutions. To receive full credit, simply explain in your own words how you approached the problems and arrived at your answers.

- Submit solutions on Gradescope. LaTeX typesetting strongly encouraged.

Late Submission and Homework Grading Policy

You may take up to two late days for any problem set. These extensions are granted automatically (no need to ask permission!). We cannot accept any submissions that are more than two days late. If you have an extenuating circumstance such as a medical emergency, please reach out to the course staff.

Problem Set 0 is an exception: we cannot accept late submissions outside of extenuating circumstances.

To encourage timely submission, two bonus points (out of 100) will be granted to submissions that are on time.

Collaboration Policy

You are free (and strongly encouraged!) to collaborate with each other on Problem Sets 0–5. Please adhere to the following policies regarding collaboration and attribution.

- Individually write up problem set solutions in your own words. You may not share write-ups with your collaborators.
- List all student collaborators and consulted outside sources (e.g. textbooks, course notes) in your problem set solutions.
- You may not collaborate with other students on Problem Set 6.

AI Use Policy

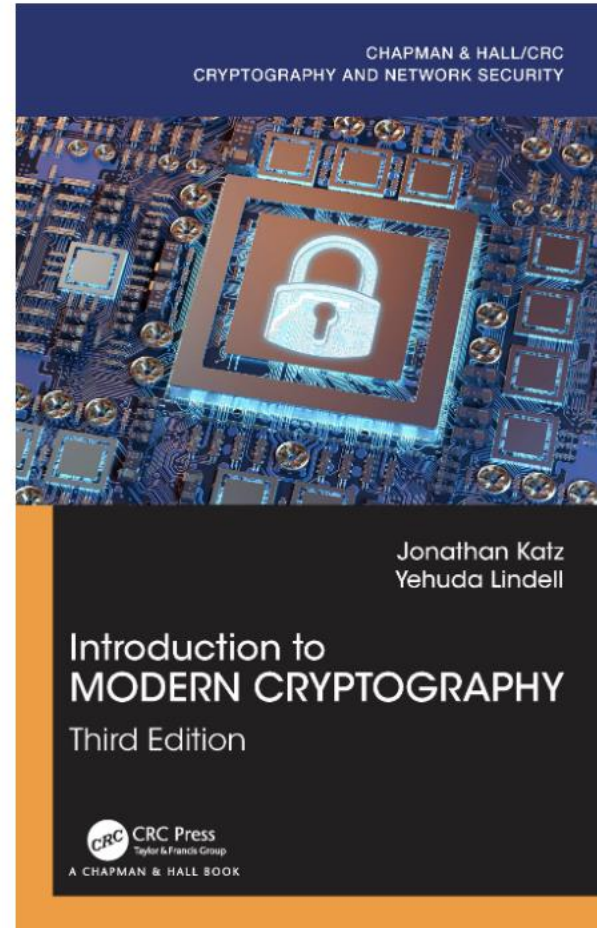
AI tools can be useful learning aids, and you are welcome to use them to explore and review general course material. You may also use AI to generate visual aids, such as diagrams, in your assignments. However, please follow two rules:

- Do not ask AI tools for solutions to assigned problems.
- Submit only writing that is entirely your own; do not submit any AI-generated text.

Feel free to reach out with any questions!

Optional Textbook

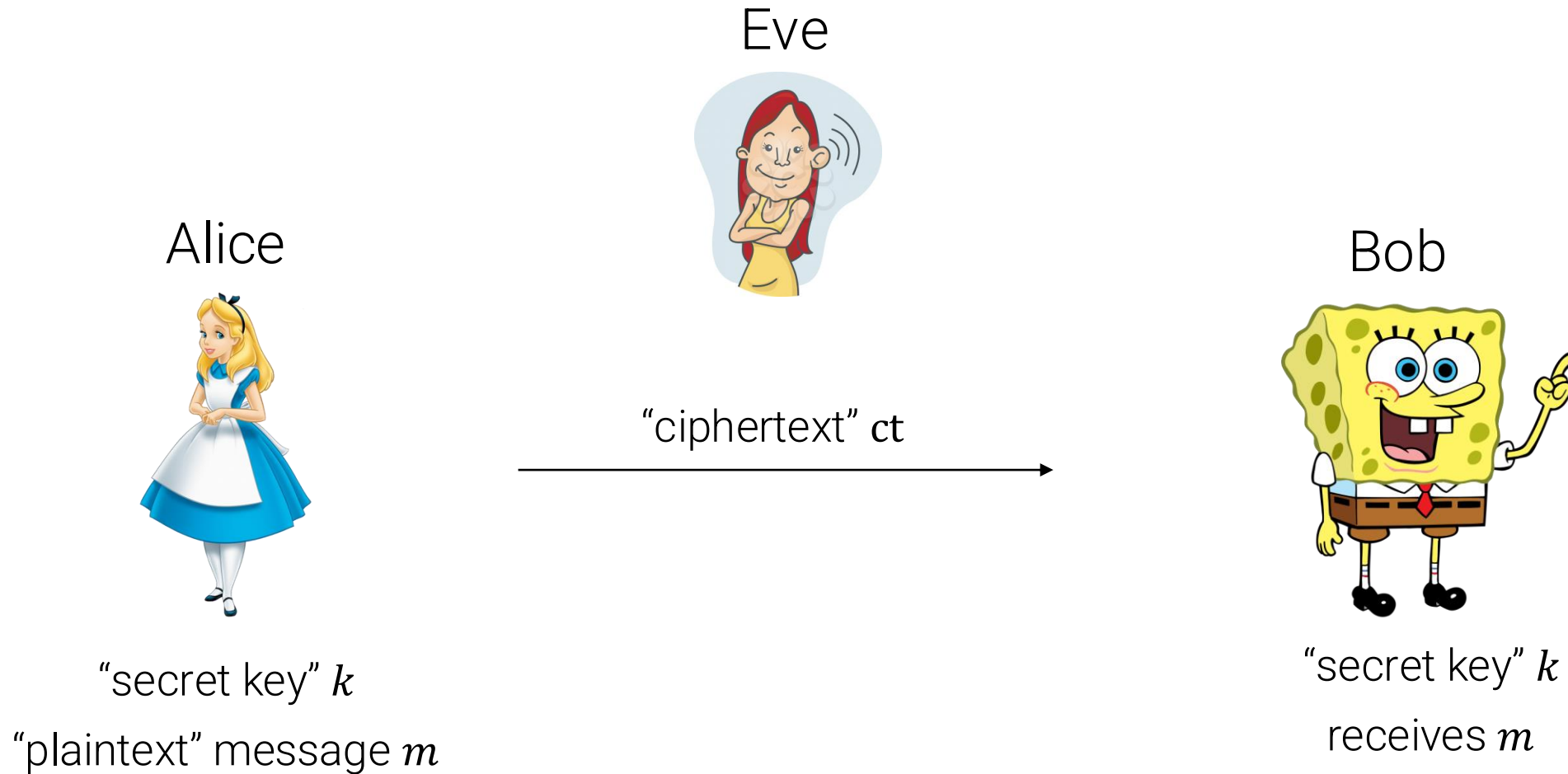
“Katz-Lindell”



Please interrupt me if you are confused!

Part I: Secret-Key Encryption and Authentication

Secret-Key Encryption



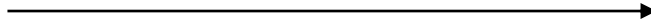
Key point: must leverage some sort of **information asymmetry** between Bob and Eve

Secret-Key Encryption: Syntax



k, m

$$ct = \text{Enc}(k, m)$$



k

$$m' = \text{Dec}(k, ct)$$

Definition: A secret-key encryption scheme (Enc, Dec) consists of two algorithms:

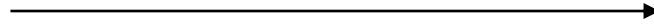
- $\text{Enc}(k, m)$ takes a key $k \in \mathcal{K}$ and message $m \in \mathcal{M}$ and outputs a ciphertext $ct \in \mathcal{C}$
- $\text{Dec}(k, ct)$ takes as input a key and ciphertext and outputs a message m'

Secret-Key Encryption: Syntax



k, m

$$ct = \text{Enc}(k, m)$$



k

$$m' = \text{Dec}(k, ct)$$

Definition: A secret-key encryption scheme (Enc, Dec) consists of two algorithms:

- $\text{Enc}(k, m; r)$ takes a key $k \in \mathcal{K}$ and message $m \in \mathcal{M}$ and outputs a ciphertext $ct \in \mathcal{C}$
- $\text{Dec}(k, ct)$ takes as input a key and ciphertext and outputs a message m'

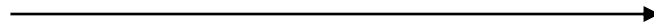
Disclaimer: encryption can either be **deterministic** or **randomized**

Secret-Key Encryption: Syntax



k, m

$$ct = \text{Enc}(k, m)$$



k

$$m' = \text{Dec}(k, ct)$$

Definition: A secret-key encryption scheme (Enc, Dec) consists of two algorithms:

- $\text{Enc}(k, m)$ takes a key $k \in \mathcal{K}$ and message $m \in \mathcal{M}$ and outputs a ciphertext $ct \in \mathcal{C}$
- $\text{Dec}(k, ct)$ takes as input a key and ciphertext and outputs a message m'

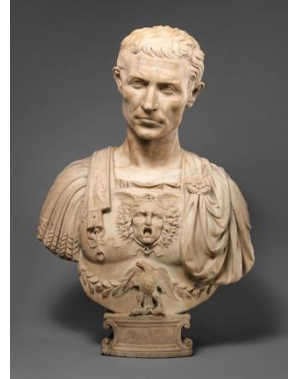
A secret-key encryption scheme (Enc, Dec) is **correct** if for all $k \in \mathcal{K}, m \in \mathcal{M}$

$$\text{Dec}(k, \text{Enc}(k, m)) = m$$

Secret-Key Encryption

Historical examples: <https://cryptii.com/>

- Caesar cipher: map every character ch of the English (or Latin) alphabet to $ch + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$



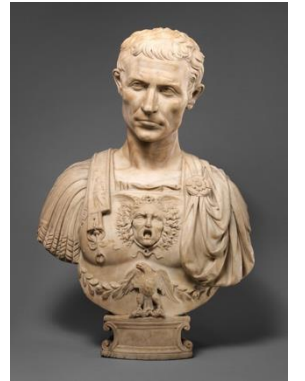
VIEW Text ▾ Veni vidi vici	ENCODE DECODE Caesar cipher ▾ SHIFT - 3 a→d + ALPHABET abcdefghijklmnopqrstuvwxyz CASE STRATEGY Ignore case ▾ FOREIGN CHARS Include Ignore → Encoded 14 chars	VIEW Text ▾ yhql ylg l ylf l
---	--	---

Secret-Key Encryption

Historical examples: <https://cryptii.com/>

- Caesar cipher: map every character ch of the English (or Latin) alphabet to $ch + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$
 - Suppose you **know** that a ciphertext ct is encrypted via Caesar. Can you, Eve, break the scheme?

Brute force attack: compute $\text{Dec}(k', ct)$ for all 26 possible k' .

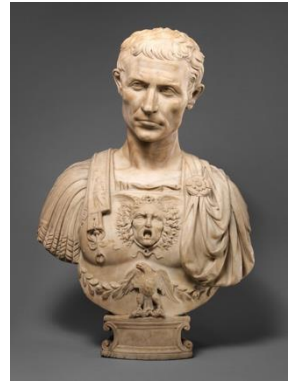


“Yhql ylgf ylf!”

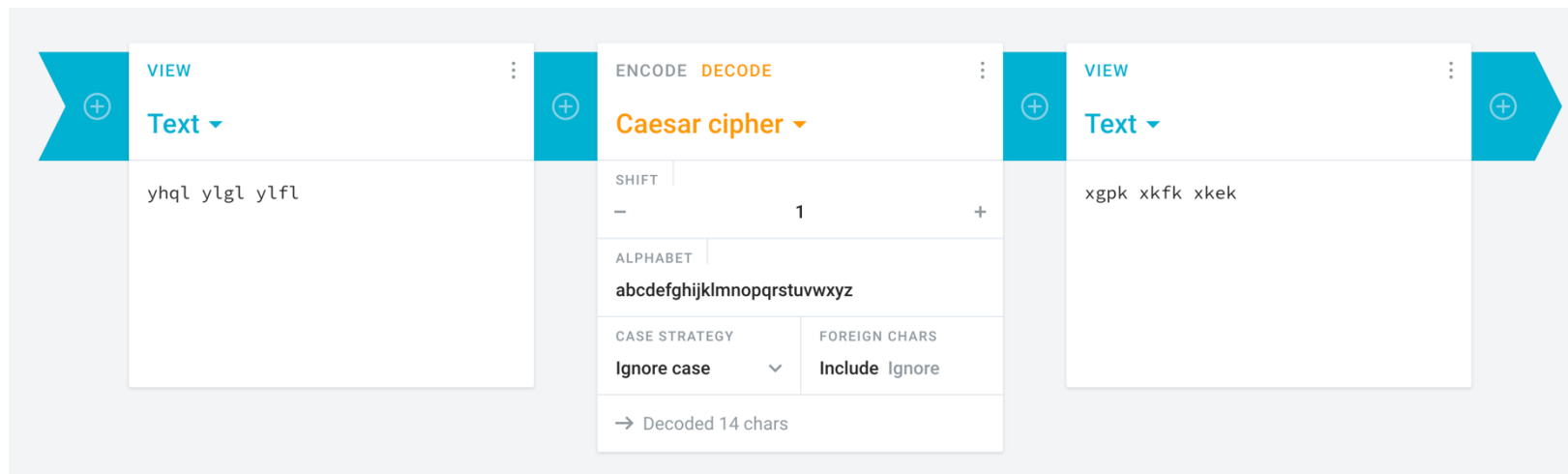
Secret-Key Encryption

Historical examples: <https://cryptii.com/>

- Caesar cipher: map every character ch of the English (or Latin) alphabet to $ch + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$
- Suppose you **know** that a ciphertext ct is encrypted via Caesar. Can you, Eve, break the scheme?



“Yhql ylgI ylfI”

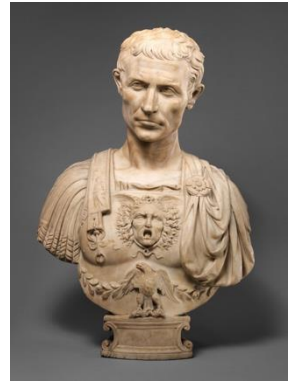


The screenshot shows the Cryptii website's Caesar cipher tool. It has three main sections: a left input panel, a central configuration panel, and a right output panel. The left panel is labeled 'Text' and contains the input 'yhql ylgI ylfI'. The central panel is labeled 'Caesar cipher' and shows a 'SHIFT' of 1, an 'ALPHABET' of 'abcdefghijklmnopqrstuvwxyz', a 'CASE STRATEGY' of 'Ignore case', and 'FOREIGN CHARS' set to 'Include Ignore'. The right panel is labeled 'Text' and contains the output 'xgpk xkfk xkek'. At the bottom of the central panel, it says '→ Decoded 14 chars'.

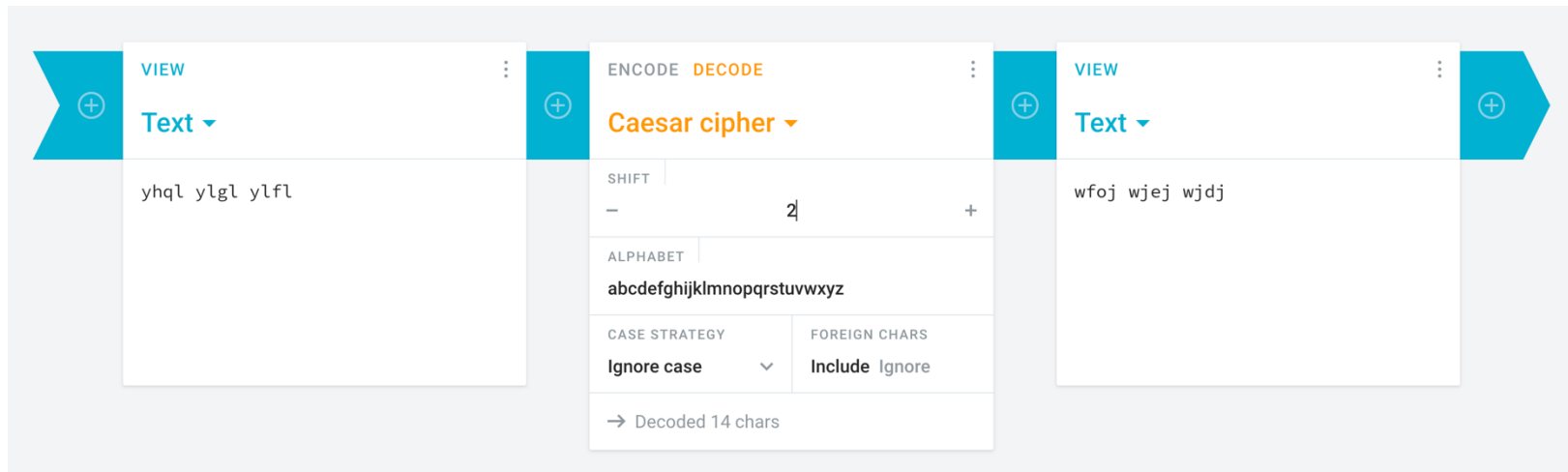
Secret-Key Encryption

Historical examples: <https://cryptii.com/>

- Caesar cipher: map every character ch of the English (or Latin) alphabet to $ch + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$
- Suppose you **know** that a ciphertext ct is encrypted via Caesar. Can you, Eve, break the scheme?



“Yhql ylgf ylfI”



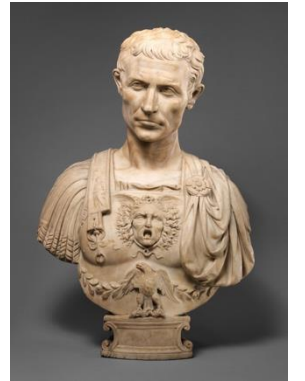
The screenshot shows the Cryptii website's Caesar cipher tool. It has three main sections: a left panel for the input text, a central control panel, and a right panel for the output text.

- Left Panel:** Labeled "VIEW" and "Text". It contains the input text "yhql ylgf ylfI".
- Central Panel:** Labeled "ENCODE" and "DECODE". It has a dropdown menu set to "Caesar cipher". Below this, there are controls for "SHIFT" (set to 2), "ALPHABET" (set to "abcdefghijklmnopqrstuvwxyz"), "CASE STRATEGY" (set to "Ignore case"), and "FOREIGN CHARS" (set to "Include"). At the bottom, it says "Decoded 14 chars".
- Right Panel:** Labeled "VIEW" and "Text". It contains the output text "wfoj wjej wjdj".

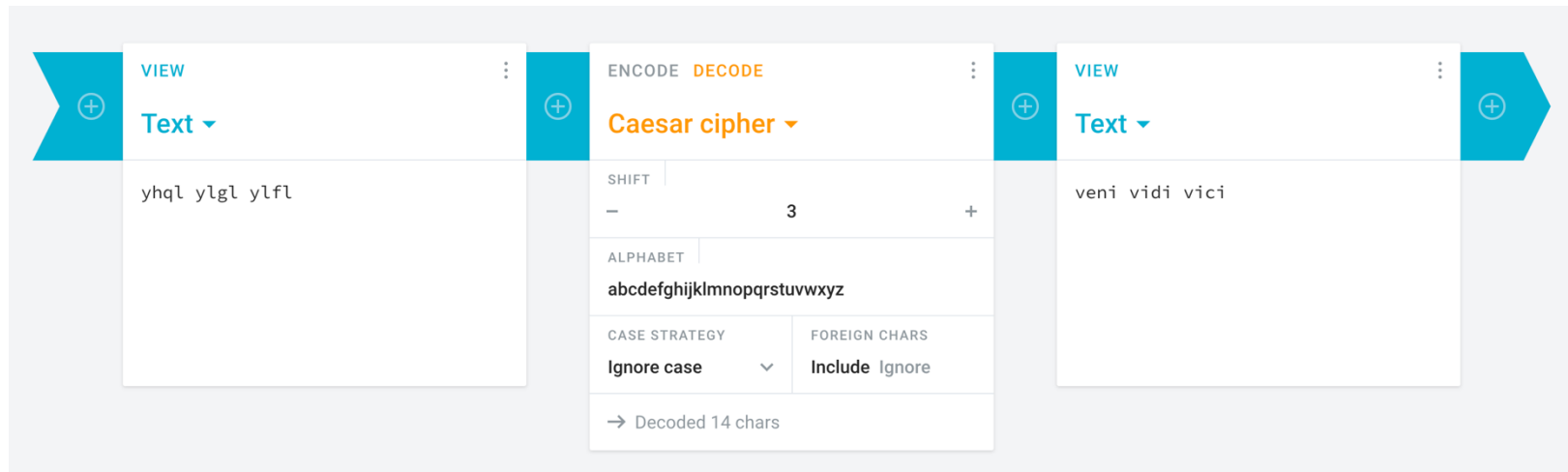
Secret-Key Encryption

Historical examples: <https://cryptii.com/>

- Caesar cipher: map every character ch of the English (or Latin) alphabet to $ch + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$
- Suppose you **know** that a ciphertext ct is encrypted via Caesar. Can you, Eve, break the scheme?



“Yhql ylgf ylfI”

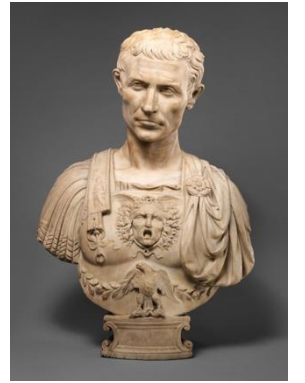


The screenshot shows the Cryptii website's Caesar cipher tool interface. It consists of three main panels. The left panel, labeled 'Text', contains the ciphertext 'yhql ylgf ylfI'. The middle panel, labeled 'Caesar cipher', has a 'SHIFT' of 3 and an 'ALPHABET' of 'abcdefghijklmnopqrstuvwxyz'. It also has options for 'CASE STRATEGY' (set to 'Ignore case') and 'FOREIGN CHARS' (set to 'Include'). The right panel, labeled 'Text', contains the plaintext 'veni vidi vici'. At the bottom of the middle panel, it says '→ Decoded 14 chars'.

Secret-Key Encryption

Historical examples: <https://cryptii.com/>

- Caesar cipher: map every character ch of the English (or Latin) alphabet to $ch + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$
 - Suppose you **know** that a ciphertext ct is encrypted via Caesar. Can you, Eve, break the scheme?



"Yhql ylgf ylf"

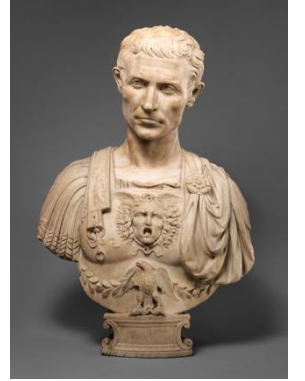
Brute force attack: compute $\text{Dec}(k', ct)$ for all 26 possible k' .

If m is an English language phrase or sentence, most likely only one makes any sense.

Secret-Key Encryption

Historical examples: <https://cryptii.com/>

- Caesar cipher: map every character ch of the English (or Latin) alphabet to $ch + k$ (with wrap-around). $k \in \mathbb{Z}, 0 \leq k \leq 25$
 - Suppose you **know** that a ciphertext ct is encrypted via Caesar. Can you, Eve, break the scheme?



“Yhql ylgI ylfI”

The only thing protecting you when you use a Caesar cipher is **obscurity** (Eve might not know you are using a Caesar cipher).

“Security through obscurity” -- unreliable

Secret-Key Encryption

Historical examples: <https://cryptii.com/>

- Vigenère cipher: key is a character string $k_1 \dots k_n$ (often a word)
 - Combines n Caesar ciphers by alternating between the k_i , letter by letter

The screenshot shows the Cryptii Vigenère cipher tool interface. It consists of three main panels: a left input panel, a central configuration panel, and a right output panel.

- Left Panel:** Labeled "VIEW" and "Text". It contains the plaintext: "We attempt to hide a message with the vigenere cipher."
- Central Panel:** Labeled "ENCODE" and "DECODE". It is configured for "Vigenère cipher".
 - VARIANT:** Standard Vigenère cipher
 - KEY:** secret
 - KEY MODE:** Repeat
 - ALPHABET:** abcdefghijklmnopqrstuvwxyz
 - CASE STRATEGY:** Ignore case
 - FOREIGN CHARS:** Include Ignore
 - Result:** → Encoded 55 chars
- Right Panel:** Labeled "VIEW" and "Text". It contains the ciphertext: "oi ckxxetv ks aahg r qxkwcxi paxj klx nmivrxji eztawv."

Secret-Key Encryption

Historical examples: <https://cryptii.com/>

- Vigenère cipher: key is a character string $k_1 \dots k_n$ (often a word)
 - Combines n Caesar ciphers by alternating between the k_i , letter by letter

Suppose you **know** that a ciphertext ct is encrypted via Vigenère. Can you, Eve, break the scheme?

Much more challenging! Brute force no longer works if n is large (26^n keys!) .

Secret-Key Encryption

Historical examples: <https://cryptii.com/>

- Vigenère cipher: key is a character string $k_1 \dots k_n$ (often a word)
 - Combines n Caesar ciphers by alternating between the k_i , letter by letter

The screenshot shows the Cryptii Vigenère cipher tool interface. It consists of three main panels: a left panel for the plaintext, a central panel for cipher settings, and a right panel for the ciphertext.

- Left Panel:** Labeled "VIEW" and "Text". It contains the plaintext: "We attempt to hide a message with the vigenere cipher. The longer the message is, the easier it should be to find the key."
- Central Panel:** Labeled "ENCODE" and "DECODE". It is titled "Vigenère cipher". It includes several settings:
 - VARIANT:** Standard Vigenère cipher
 - KEY:** secret
 - KEY MODE:** Repeat
 - ALPHABET:** abcdefghijklmnopqrstuvwxyz
 - CASE STRATEGY:** Ignore case
 - FOREIGN CHARS:** Include IgnoreAt the bottom, it shows "→ Encoded 122 chars".
- Right Panel:** Labeled "VIEW" and "Text". It contains the ciphertext: "oi ckxsetv ks aahg r qxkwcxi paxj klx nmivrxji eztawv. vyi egrivv mzi ovwlskg zw, mzi grwbwv kk wagynu fx ls hzrw llg bir."

Secret-Key Encryption

Historical examples: <https://cryptii.com/>

- Substitution cipher: key is a permutation π of the alphabet. Map every character ch to $\pi(ch)$. $26! \approx 4 \cdot 10^{26}$ keys, cannot brute force by hand.

The screenshot displays the Cryptii website interface for an alphabetical substitution cipher. The interface is divided into three main sections: a left panel for the plaintext, a central panel for the cipher configuration, and a right panel for the ciphertext.

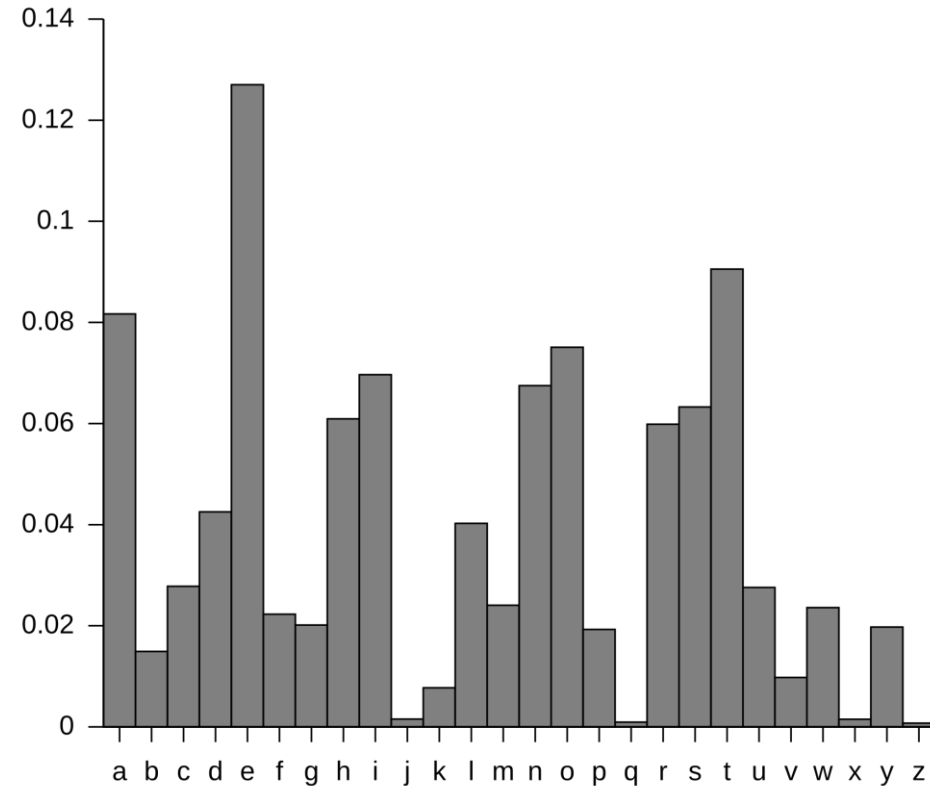
Left Panel (Text): Contains the text: "We attempt to hide a message with the substitution cipher. The longer the message is, the easier it should be to find the key."

Central Panel (Configuration): Features a toggle for "Alphabetical substitution" and a "VIEW" button. Below this, it shows the "PLAINTEXT ALPHABET" as "abcdefghijklmnopqrstuvwxyz" and the "CIPHERTEXT ALPHABET" as "zyxwvutsrqponmlkjihgfedcba". There are also options for "CASE STRATEGY" (set to "Ignore case") and "FOREIGN CHARS" (with "Include" and "Ignore" options). At the bottom, it indicates "→ Encoded 126 chars".

Right Panel (Text): Contains the resulting ciphertext: "dv zggvnkg gl srwv z nvhhztv drgs gsv hfyhgrgfggrlm xrksvi. gsv olmtvi gsv nvhhztv rh, gsv vzhrvi rg hslfow yv gl urmw gsv pvb."

Breaking Substitution Ciphers

This is a graph of the frequency of each letter in written English.



Given a sufficiently long ciphertext, one can generate this graph (“frequency analysis”) and greatly narrow down what π is.

Breaking the Vigenère Cipher

Vigenère cipher: key is a character string $k_1 \dots k_n$ (often a word)

- Combines n Caesar ciphers by alternating between the k_i , letter by letter

Attack:

1. **Brute force** over the possible **key lengths** (e.g. start from $n = 1$ and count up)
2. Divide the ciphertext characters into n groups (following the Vigenère pattern)
3. Perform **frequency analysis** on the n groups separately!

Breaking the Vigenère Cipher

- Our attack did not make use of the fact that each of the n parts are encrypted with a Caesar cipher; would have worked even if they were substitution ciphers.
- This breaks the more general class of “poly-alphabetic ciphers”
- However, this does require that Eve can obtain a sufficiently long ciphertext encrypting “natural language.”

Secret-Key Encryption

Enigma cipher: extremely complex variant of poly-alphabetic cipher.

- Utilized by the Axis powers in WWII
- Theoretically broken given enormously long messages, but their protocols called for changing keys too frequently for this to happen.



VIEW
Text

We attempt to hide a message with the Enigma cipher. The implementation details of the Enigma cipher are so complex that we will not cover them in this class. It was far more difficult to break Enigma compared to the ciphers discussed so far.

ENCODE DECODE
Enigma machine

MODEL
Enigma M3

REFLECTOR
UKW B

ROTOR 1	POSITION	RING
IV	- 14 N +	- 5 E +
ROTOR 2	POSITION	RING
III	- 5 E +	- 21 U +
ROTOR 3	POSITION	RING
VII	- 23 W +	- 12 L +

PLUGBOARD

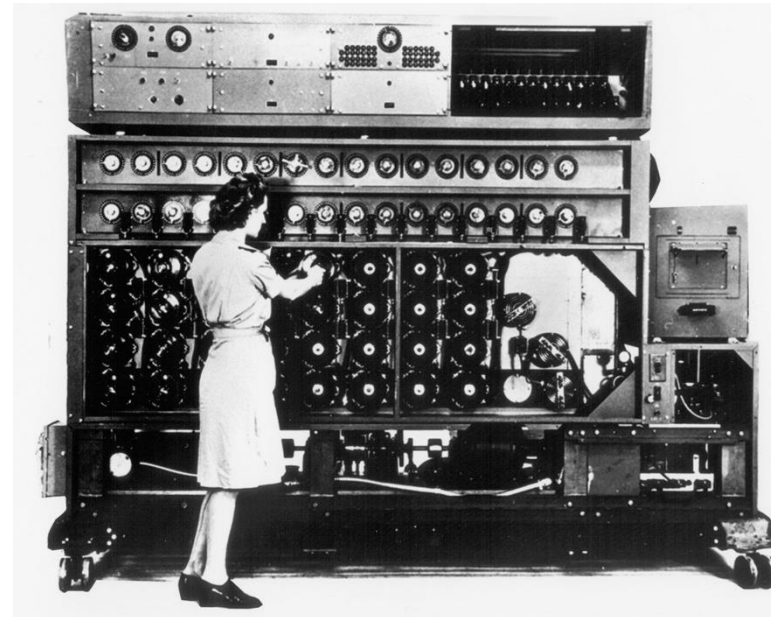
VIEW
Text

vcgyw auityp qwcfz klwmy cqquy
keudy swjlv njbtu ocfsy gxldw
dlkaf jpgsp hxdbj wemvp ugkhn
rsiwo gtcpt bhiqn gkhwf ezzpb
bfafz xbrny llvfn ubzql tvnra
eiabx deddy ncfzi aopgj lmigl
ckvnw vxbrt folxj bylfq cqlob
kzxba kvypn xbnkh yilaf t

Secret-Key Encryption

Enigma cipher: extremely complex variant of poly-alphabetic cipher.

- Broken in practice in several cycles (the Germans employed countermeasures)
- Contributing factors: **obtaining copies of the machine**, exploiting small flaws to **narrow key search space**, user error (**bad keys**), 2^{20} time computer search.



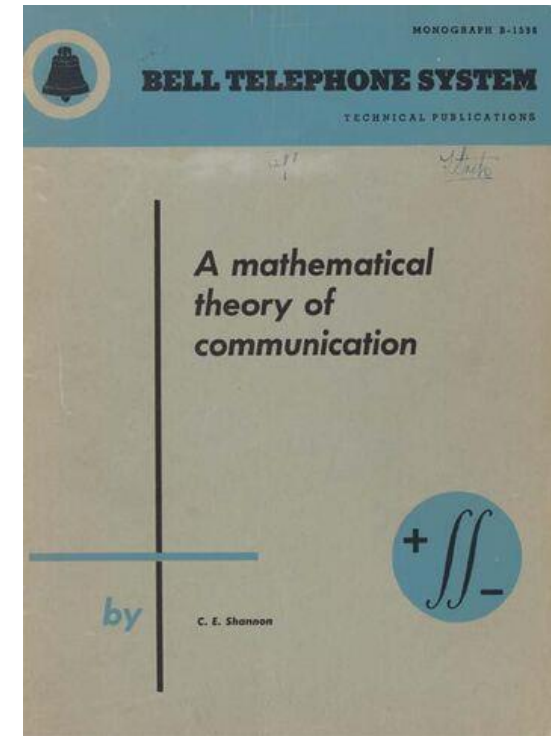
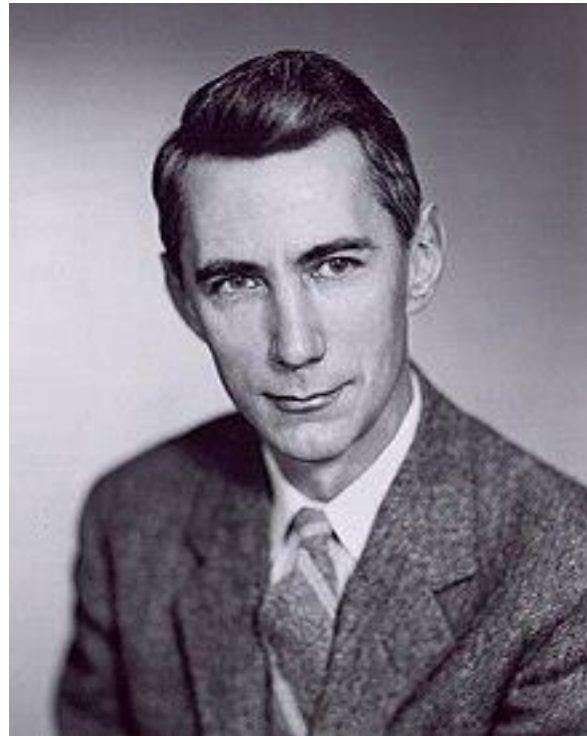
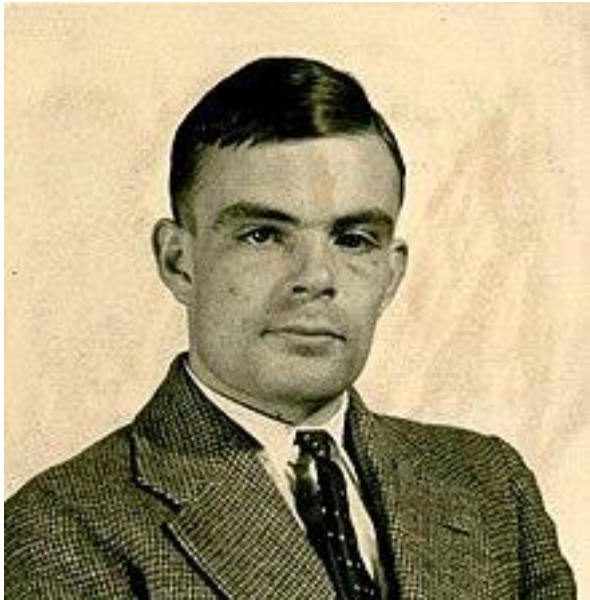
Secret-Key Encryption

Takeaways:

- It can be really hard to intuit if an encryption scheme is truly secure!
- **Security through obscurity** is a bad plan in the long run.
 - Kerckhoffs' principle (1883): the security of a cryptosystem **shouldn't rely on the secrecy of the algorithm** (only the key)
 - NSA: "The standard assumption there was that serial number one of any new device was delivered to the Kremlin."
- What we need is a formal framework for **defining and proving security**



Secret-Key Encryption



- What we need is a formal framework for **defining and proving security**

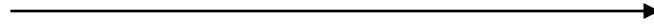
Information/probability theory

Secret-Key Encryption: Security



k, m

$$ct = \text{Enc}(k, m)$$



k

Adversary Model: Eve is “passive,” reads communication but does not interfere.

Insight: Alice and Bob should sample k **uniformly at random** from $\mathcal{K}$.

Major simplification: focus on the case of encrypting a *single message*.

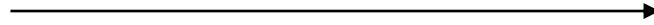
How should we define security?

Secret-Key Encryption: Security



k, m

$$ct = \text{Enc}(k, m)$$



k

Proposal 1: Scheme is secure if the key is completely unguessable.

Formally, for all $m \in \mathcal{M}$ and all functions $\text{Eve}: \mathcal{C} \rightarrow \mathcal{K}$

$$\Pr_{k \leftarrow \mathcal{K}} [\text{Eve}(\text{Enc}(k, m)) = k] = \frac{1}{|\mathcal{K}|}$$

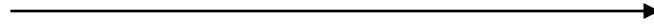
Counterexample: $\text{Enc}(k, m) = m$

Secret-Key Encryption: Security



k, m

$$ct = \text{Enc}(k, m)$$



k

Proposal 2: Scheme is secure if the **message** is completely unguessable.

What does that mean?

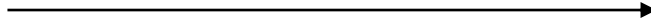
Let's handle the case of a uniformly random message.

Secret-Key Encryption: Security



k, m

$$ct = \text{Enc}(k, m)$$



k

Definition 1: a secret-key encryption scheme (Enc, Dec) is **perfectly secret for uniform messages** if for all functions $\text{Eve}: \mathcal{C} \rightarrow \mathcal{M}$

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ m \leftarrow \mathcal{M}}} [\text{Eve}(\text{Enc}(k, m)) = m] = \frac{1}{|\mathcal{M}|}$$

Perfect Secrecy

Definition 1: a secret-key encryption scheme (Enc, Dec) is **perfectly secret for uniform messages** if for all functions $\text{Eve}: \mathcal{C} \rightarrow \mathcal{M}$

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ m \leftarrow \mathcal{M}}} [\text{Eve}(\text{Enc}(k, m)) = m] = \frac{1}{|\mathcal{M}|}$$

Perfect Secrecy

Definition 1': a secret-key encryption scheme (Enc, Dec) is **perfectly secret for uniform messages** if for all messages $m^* \in \mathcal{M}$ and all ciphertexts $ct \in \mathcal{C}$

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ m \leftarrow \mathcal{M}}} [m = m^* \mid \text{Enc}(k, m) = ct] = \frac{1}{|\mathcal{M}|}$$

This is equivalent to Definition 1. Why?

The best strategy for Eve given ct is to guess the message with the **highest conditional probability** of occurring.

Perfect Secrecy

Definition 1': a secret-key encryption scheme (Enc, Dec) is **perfectly secret for uniform messages** if for all messages $m^* \in \mathcal{M}$ and all ciphertexts $ct \in \mathcal{C}$

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ m \leftarrow \mathcal{M}}} [m = m^* \mid \text{Enc}(k, m) = ct] = \frac{1}{|\mathcal{M}|}$$

But my messages aren't (uniformly) random!

Perfect Secrecy

Definition 2: a secret-key encryption scheme (Enc, Dec) is **perfectly semantically secure** if for all random variables M over messages, all messages $m^* \in \mathcal{M}$ and all $ct \in \mathcal{C}$

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ M}} [M = m^* \mid \text{Enc}(k, M) = ct] = \Pr_M [M = m^*]$$



k, M

$C = \text{Enc}(k, M)$



k

Perfect Secrecy

Definition 2: a secret-key encryption scheme (Enc, Dec) is **perfectly semantically secure** if for all random variables M over messages, all messages $m^* \in \mathcal{M}$ and all $ct \in \mathcal{C}$

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ M}} [M = m^* \mid \text{Enc}(k, M) = ct] = \Pr_{\substack{M}} [M = m^*]$$

Theorem: a secret-key encryption scheme is perfectly semantically secure if and only if it is perfectly secret for uniform messages!

Perfect Secrecy

Definition 3: a secret-key encryption scheme (Enc, Dec) is **perfectly indistinguishable** if for all message pairs (m_0, m_1)

$$\text{Enc}(k, m_0) \equiv \text{Enc}(k, m_1)$$



k, m_b

$$\text{ct} = \text{Enc}(k, m_b)$$



k

Perfect Secrecy

Definition 3: a secret-key encryption scheme (Enc, Dec) is **perfectly indistinguishable** if for all message pairs (m_0, m_1)

$$\text{Enc}(k, m_0) \equiv \text{Enc}(k, m_1)$$

Theorem: a secret-key encryption scheme is perfectly indistinguishable if and only if it is perfectly secret on uniform messages/perfectly semantically secure.

Perfect Secrecy

Theorem: Definitions 1, 2, and 3 are equivalent

Proof: $2 \rightarrow 1 \rightarrow 3 \rightarrow 2$

Perfect Secrecy

Theorem: Definitions 1, 2, and 3 are equivalent

Proof: $2 \rightarrow 1 \rightarrow 3 \rightarrow 2$

Take M to be the uniform distribution

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ M}} [M = m \mid \text{Enc}(k, M) = \text{ct}] = \Pr_M [M = m] = \frac{1}{|\mathcal{M}|}$$

Since Eve is only given $\text{Enc}(k, M)$, she has no hope.

Perfect Secrecy

Theorem: Definitions 1, 2, and 3 are equivalent

Proof: $2 \rightarrow 1 \rightarrow 3 \rightarrow 2$

Proof by contradiction. Suppose that $\exists(m_0, m_1)$ such that

$$\text{Enc}(k, m_0) \not\equiv \text{Enc}(k, m_1)$$

Then, $\exists \text{ ct}^*$ such that

$$\Pr_{k \leftarrow \mathcal{K}} [\text{Enc}(k, m_0) = \text{ct}^*] > \Pr_{k \leftarrow \mathcal{K}} [\text{Enc}(k, m_1) = \text{ct}^*]$$

Perfect Secrecy

Theorem: Definitions 1, 2, and 3 are equivalent

Proof: $2 \rightarrow 1 \rightarrow 3 \rightarrow 2$

Want to show:
$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ m \leftarrow \mathcal{M}}} [\text{Eve}(\text{Enc}(k, m)) = m] > \frac{1}{|\mathcal{M}|} \quad \text{for some Eve.}$$

Eve(ct):

- If $\text{ct} \neq \text{ct}^*$, guess uniformly from $\mathcal{M}$.
- If $\text{ct} = \text{ct}^*$, guess uniformly from $\mathcal{M}$ but substitute m_0 for m_1
- This is strictly better than uniform guess.

Perfect Secrecy

Theorem: Definitions 1, 2, and 3 are equivalent

Proof: $2 \rightarrow 1 \rightarrow 3 \rightarrow 2$

Want to show:

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ M}} [M = m^* \mid \text{Enc}(k, M) = \text{ct}] = \Pr_{\substack{M}} [M = m^*]$$

Perfect Secrecy

Theorem: Definitions 1, 2, and 3 are equivalent

Proof: $2 \rightarrow 1 \rightarrow 3 \rightarrow 2$

Want to show:

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ M}} [M = m^* \wedge \text{Enc}(k, M) = \text{ct}] = \Pr_{\substack{M}} [M = m^*] \cdot \Pr_{\substack{k, M}} [\text{Enc}(k, M) = \text{ct}]$$

Perfect Secrecy

Theorem: Definitions 1, 2, and 3 are equivalent

Proof: $2 \rightarrow 1 \rightarrow 3 \rightarrow 2$

We know:

$$\Pr_{\substack{k \leftarrow \mathcal{K} \\ M}} [M = m^* \wedge \text{Enc}(k, M) = \text{ct}] = \Pr_{\substack{M \\ k}} [M = m^*] \cdot \Pr_k [\text{Enc}(k, m^*) = \text{ct}]$$

Claim: $\Pr_k [\text{Enc}(k, m^*) = \text{ct}] = \Pr_{k, M} [\text{Enc}(k, M) = \text{ct}]$

Proof: $\forall m, \Pr_k [\text{Enc}(k, m^*) = \text{ct}] = \Pr_k [\text{Enc}(k, m) = \text{ct}]$

Perfect secrecy seems like a nice security notion!

It would be a shame if something were to happen to it...

COS 433: Cryptography

Instructor: Alex Lombardi, Office Hours Monday 4:15pm

TAs: Lacey Delucia, Fangqi Dong, Frederick Qiu (OH TBD)

Email (use this to reach us): cos433staff@princeton.edu

Course Website:



princeton-cos-433.github.io